

Documento Técnico de Código Fuente y Algoritmos de Optimización

Sistema de Asistencia Inteligente de Admisiones: "Nova OS '97"

Evidencia de Producto 1 — Norma SENA 220501096

- **Programa de Formación:** Análisis y Desarrollo de Software (ADSO)
- **Norma de Competencia:** 220501096 — *Desarrollar solución de software de acuerdo con especificaciones de diseño y marcos de referencia.*
- **Candidato / Aprendiz:** Brandon Jose Carranza Rangel
- **Documento de Identidad:** C.C. 1007892884
- **Organización Beneficiaria:** Nova Idiomas Colombia
- **Fecha de Elaboración:** 2026-09-02 (Zona Horaria: America/Bogota)
- **Versión del Código:** v2.6.0 (27/27 Tests Aprobados en Pytest)

1. Organización del Proyecto y Estructura de Directorios

El proyecto implementa una arquitectura desacoplada de monorepo limpio (*Clean Architecture*), separando el núcleo de computación de inteligencia artificial en Python del cliente web en TypeScript:

```
NovaVice_os97/
├── backend/
│   ├── data/
│   │   ├── documents/
│   │   ├── chroma_db/
│   │   └── escalations.json
│   ├── src/
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── api/
│   │   │   ├── routes.py
│   │   │   └── schemas.py
│   │   ├── core/
│   │   │   ├── guardrails.py
│   │   │   ├── navigation.py
│   │   │   ├── cache.py
│   │   │   ├── dispatcher.py
│   │   │   ├── memory.py
│   │   │   ├── metrics.py
│   │   │   └── opencode_client.py
│   │   └── rag/
│   │       ├── bm25.py
│   │       ├── hybrid_retriever.py
│   │       ├── vector_store.py
│   │       ├── ingestion.py
│   │       ├── engine.py
│   │       └── prompt_templates.py
│   ├── tests/
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── components/
│   │   │   │   ├── PixiParticleBackground.tsx
│   │   │   │   ├── ChatContainer.tsx
│   │   │   │   ├── ChatInput.tsx
│   │   │   │   ├── Header.tsx
│   │   │   │   └── MetricsModal.tsx
│   │   │   └── lib/
│   │   ├── package.json
│   │   └── tailwind.config.ts
├── installer.py
├── run.py
├── install.bat / install.sh
└── start.bat / start.sh
```

Núcleo Backend en Python Puro (FastAPI)
Base de conocimiento oficial y persistencia
83 documentos normativos oficiales en Markdown
Vector Store persistente local (ONNX embeddings)
Almacén de tickets de derivación a asesores
Código fuente modular
Entrypoint de FastAPI, middlewares y arranque
Variables de entorno tipadas con Pydantic Settings
Capa de controladores y rutas REST
Endpoints /chat, /stream, /metrics, /health, /escalate
Contratos I/O Pydantic v2
Servicios de seguridad, navegación y caché
Filtro pre-flight contra prompt injection
Máquina de estados con 4 pilares y 85 sinónimos
Caché dual: exacta SHA-256 y semántica coseno
Generador de tickets ESC-YYYYMMDD-XXXX
Estado conversacional del aspirante
Bus de telemetría y exportador de métricas
Cliente HTTP hacia OpenCode Daemon (:4096)
Subsistema de recuperación aumentada por generación
Motor léxico Okapi BM25 en Python con lematización
Fusión Reciprocal Rank Fusion (RRF, k=60)
Adaptador de ChromaDB local y embeddings
Pipeline de chunking con solapamiento (overlap)
Orquestador maestro del flujo RAG
Plantillas de prompt institucionales
Suite de pruebas automatizadas (27 tests en Pytest)
Dependencias de producción y testing

Capa de Presentación Web (Next.js 15)
Next.js App Router (layout.tsx, page.tsx, globals.css)
Componentes UI reutilizables
Lienzo WebGL con aceleración GPU (PixiJS)
Renderizado de Markdown GFM con componentes
Entrada de texto y atajos contextuales
Barra retro con reloj y conmutador CRT
Modal de auditoría de latencia y costos
Clientes API fetch y tipos TypeScript
Dependencias de Node.js y frameworks
Configuración de estilos retro synthwave

Instalador automatizado multiplataforma
Supervisor multi-proceso sincronizado
Lanzadores de instalación en un clic
Lanzadores de ejecución en un clic

2. Tecnologías y Frameworks Utilizados

Componente	Tecnología	Versión	Rol en el Sistema
Backend Core	Python	3.10+	Lenguaje base para servicios RAG, cálculo matemático y lógica de negocio.
API Gateway	FastAPI	0.115+	Framework web asíncrono para endpoints REST de baja latencia.
Validación I/O	Pydantic	2.9+	Tipado estático y validación de contratos de entrada y salida.
Vector Store	ChromaDB	0.5+	Almacenamiento y búsqueda de similitud coseno vectorial en local.
Embeddings	ONNX Runtime	1.18+	Inferencia de <code>all-MiniLM-L6-v2</code> (384 dimensiones) en CPU sin costo de API.
Cliente HTTP	HTTPX	0.27+	Conexiones asíncronas con pool persistente hacia el servidor de razonamiento.
Frontend Core	Next.js	15.5+	Framework React con Server Components y App Router.
Interfaz Declarativa	React	19.0+	Renderizado de vistas reactivas.
Gráficos WebGL	PixiJS	8.8+	Renderizado 2D acelerado por GPU para el fondo de partículas.
Estilos UI	Tailwind CSS	3.4+	Sistema de utilidades CSS con temática retro y filtro CRT.
Markdown GFM	ReactMarkdown + RemarkGFM	9.0+	Renderizado nativo de texto enriquecido, tablas y citas sin sintaxis residual.
Testing	Pytest	8.3+	Suite automatizada de pruebas unitarias e integrales (27/27 passed).

3. Algoritmos Matemáticos y Lógica de Optimización

Para maximizar el rendimiento computacional, eliminar alucinaciones y lograr respuestas en sub-30ms, el sistema implementa seis algoritmos matemáticos fundamentales:

3.1 Algoritmo de Búsqueda Léxica Okapi BM25 con Saturación de Término

El algoritmo Okapi BM25 puntúa la relevancia de un documento D respecto a una consulta $Q = q_1, q_2, \dots, q_n$ considerando la saturación de frecuencia y la penalización por longitud:

$$\text{Score}_{\text{BM25}}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1} \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)$$

donde:

- $f(q_i, D)$ es la frecuencia de aparición del término q_i en el documento D .
- $|D|$ es la longitud en tokens del fragmento evaluado y `avgdl` es la longitud promedio de los documentos del corpus.
- $\text{IDF}(q_i)$ es la frecuencia inversa de documento calculada como:

$$\text{IDF}(q_i) = \ln \left(1 + \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} \right)$$

donde N es el total de fragmentos ($N = 252$) y $n(q_i)$ es la cantidad de fragmentos que contienen la palabra.

- Parámetros calibrados:** $k_1 = 1.5$ (saturación de frecuencia de término) y $b = 0.75$ (grado de normalización por longitud de documento).

3.2 Similitud Coseno en Espacios Vectoriales de Alta Dimensión

Para la búsqueda semántica densa, los textos se transforman en vectores continuos normalizados $\vec{u}, \vec{v} \in \mathbb{R}^{384}$. La similitud de ángulo entre la consulta y los fragmentos institucionales se determina mediante:

$$\text{Sim}_{\cos}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\|_2 \|\vec{v}\|_2} = \frac{\sum_{i=1}^{384} u_i v_i}{\sqrt{\sum_{i=1}^{384} u_i^2} \sqrt{\sum_{i=1}^{384} v_i^2}}$$

Dado que los vectores producidos por el modelo ONNX se encuentran normalizados unitariamente ($\|\vec{u}\|_2 = 1$), el cálculo se reduce al producto escalar directo $\vec{u} \cdot \vec{v}$, reduciendo la complejidad computacional a $O(d)$.

3.3 Fusión de Rangos Recíprocos (Reciprocal Rank Fusion - RRF, $k = 60$)

La búsqueda densa y la búsqueda léxica generan puntuaciones en escalas incompatibles (coseno acotado en $[0, 1]$ frente a BM25 no acotado). Para combinarlas sin sesgos numéricos, se aplica RRF:

$$RRF(d) = \sum_{m \in \text{dense, bm25}} \frac{1}{k + \text{rank}_m(d)}$$

- La constante $k = 60$ suaviza el impacto de las posiciones superiores, garantizando que un documento que aparezca consistentemente en ambos rankings (ej. puesto 3 en denso y puesto 2 en léxico) supere a documentos con alta coincidencia en una sola dimensión.
- Factor de Cobertura de Tokens e Impulso:** Si la consulta contiene palabras clave del pilar temático detectado (ej. precios, horarios, sedes), se aplica una ponderación:

$$\text{Score}_{\text{final}} = \max(\text{Sim}(\text{dense}), \min(1.0, \frac{\text{BM25}}{3.0}) \times C \times 1.4)$$

donde $C = \frac{|\text{tokens}(Q) \cap \text{tokens}(d)|}{|\text{tokens}(Q)|}$ representa el ratio de cobertura de términos.

3.4 Algoritmo de Distancia Levenshtein con Programación Dinámica

Para brindar tolerancia a fallos tipográficos en entradas cortas (ej. `horaroi` por `horario`, `presio` por `precio`), se calcula la distancia de edición mínima entre dos cadenas s_1 y s_2 mediante la relación de recurrencia de Wagner-Fischer:

$$D(i, j) = \begin{cases} i & \text{si } j = 0, \\ j & \text{si } i = 0, \\ D(i-1, j-1) & \text{si } s_1[i] = s_2[j], \\ 1 + \min(D(i-1, j), D(i, j-1), D(i-1, j-1)) & \text{en otro caso.} \end{cases}$$

Se aplica un umbral estricto $D(s_1, s_2) \leq 2$ únicamente sobre palabras con longitud mayor a 4 caracteres, logrando resoluciones en microsegundos sin falsos positivos.

3.5 Caché Invalidation-Aware con Hash SHA-256 en $O(1)$

La optimización de costo y latencia se basa en un esquema jerárquico:

- Nivel 1 (Exacto):** La consulta se normaliza en minúsculas y se calcula su resumen criptográfico:

$$h = \text{SHA-256}(\text{normalizar}(Q))$$

La recuperación en la tabla hash en memoria se realiza en tiempo constante $O(1)$ entregando latencias de 18.4 ms.

- Nivel 2 (Semántico):** Se calcula la similitud coseno entre el embedding de la consulta entrante y las consultas almacenadas en caché. Si $\text{Sim}_{\text{cos}} \geq 0.88$ (para pilares) o ≥ 0.92 (para la política de descuentos), se reutiliza la respuesta sin invocar al modelo generativo.
- Invalidación Automática:** Se calcula un hash acumulativo de las fechas de modificación y tamaños de todos los archivos en `data/documents/`. Si este hash varía, la caché se purga en caliente.

4. Bloques de Código Fuente Reales Comentados

Los siguientes fragmentos representan la implementación real extraída del repositorio, cumpliendo con estándares de código limpio: ausencia total de emojis en el código y comentarios sobrios de una sola línea.

4.1 Motor Léxico Okapi BM25 con Lematizador en Español (backend/src/rag/bm25.py)

```
import math
import re
from typing import List, Dict, Tuple

class PureBM25:
    def __init__(self, k1: float = 1.5, b: float = 0.75):
        # Parametros de calibracion estandar para saturacion y longitud
        self.k1 = k1
        self.b = b
        self.inverted_index: Dict[str, Dict[str, int]] = {}
        self.doc_lengths: Dict[str, int] = {}
        self.avg_doc_len: float = 0.0
        self.corpus_size: int = 0

    def _stem(self, word: str) -> str:
        # Lematizador morfologico determinista para sufijos comunes en espanol
        w = word.lower()
        if w.endswith("ciones") and len(w) > 6:
            return w[:-6] + "cion"
        if w.endswith("dades") and len(w) > 5:
            return w[:-5] + "dad"
        if w.endswith("les") and len(w) > 4:
            return w[:-3] + "l"
        if w.endswith("es") and len(w) > 4:
            return w[:-2]
        if w.endswith("s") and not w.endswith("ss") and len(w) > 3:
            return w[:-1]
        return w

    def search(self, query: str, top_k: int = 15) -> List[Tuple[str, float]]:
        # Procesa los terminos de la consulta eliminando signos de puntuacion
        tokens = [self._stem(w) for w in re.findall(r"\w+", query.lower()) if len(w) > 2]
        scores: Dict[str, float] = {}
        for token in tokens:
            if token not in self.inverted_index:
                continue
            doc_freq = len(self.inverted_index[token])
            # Calculo del factor de frecuencia inversa de documento IDF
            idf = math.log(1.0 + (self.corpus_size - doc_freq + 0.5) / (doc_freq + 0.5))
            for doc_id, freq in self.inverted_index[token].items():
                doc_len = self.doc_lengths.get(doc_id, self.avg_doc_len)
                # Formula matematica Okapi BM25 con normalizacion por longitud
                denom = freq + self.k1 * (1.0 - self.b + self.b * (doc_len / self.avg_doc_len))
                score = idf * (freq * (self.k1 + 1.0)) / denom
                scores[doc_id] = scores.get(doc_id, 0.0) + score
        # Ordena descendente y extrae los mejores candidatos
        ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)
        return ranked[:top_k]
```

4.2 Fusión Reciprocal Rank Fusion (RRF) (backend/src/rag/hybrid_retriever.py)

```
from typing import List, Dict, Any

class HybridRetriever:
    def __init__(self, rrf_k: int = 60):
        # Constante k de suavizado para mitigacion del sesgo de posicion
        self.rrf_k = rrf_k

    def retrieve(self, query: str, top_k: int = 4, candidate_k: int = 15) -> List[Dict[str, Any]]:
        # Ejecuta la busqueda densa en ChromaDB y la busqueda lexica en BM25
        dense_results = self.vector_store.query(query, top_k=candidate_k)
        bm25_results = self.bm25_index.search(query, top_k=candidate_k)

        rrf_scores: Dict[str, float] = {}
        # Acumula puntajes reciprocos para los candidatos densos
        for rank, item in enumerate(dense_results):
            doc_id = item["id"]
            rrf_scores[doc_id] = rrf_scores.get(doc_id, 0.0) + (1.0 / (self.rrf_k + rank + 1))

        # Acumula puntajes reciprocos para los candidatos de BM25
        for rank, (doc_id, bm_score) in enumerate(bm25_results):
            rrf_scores[doc_id] = rrf_scores.get(doc_id, 0.0) + (1.0 / (self.rrf_k + rank + 1))

        # Ordena candidatos consolidados por su puntuacion de fusion
        sorted_candidates = sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)
        final_chunks = []
        for doc_id, score in sorted_candidates[:top_k]:
            chunk_data = self._resolve_chunk(doc_id)
            chunk_data["rrf_score"] = round(score, 5)
            final_chunks.append(chunk_data)
        return final_chunks
```

4.3 Filtro de Seguridad Pre-Flight (backend/src/core/guardrails.py)

```
import re
from typing import Tuple

class PreFlightGuardrails:
    # Patrones regex compilados para intercepcion de prompt injections y jailbreaks
    INJECTION_PATTERNS = [
        r"ignore\s+(previous|all)\s+instructions",
        r"olvida\s+(todas\s+las\tus)\s+instrucciones",
        r"reveal\s+(system\s+prompt|secret)",
        r"act\s+as\s+an?\s+(admin|hacker)",
        r"system\s*:\s*override",
        r"mode\s*:\s*developer",
        r"beca\s+(100%|total\s+gratis|gratuita)"
    ]

    def validate_query(self, query: str) -> Tuple[bool, str]:
        # Valida longitud maxima para prevenir denegacion de servicio por memoria
        if len(query) > 1000:
            return False, "La consulta excede la longitud maxima permitida de 1000 caracteres."
        # Inspecciona contra patrones de inyeccion
        for pattern in self.INJECTION_PATTERNS:
            if re.search(pattern, query, re.IGNORECASE):
                return False, "Patron de consulta no autorizado detectado por el protocolo de seguridad."
        return True, ""

    def evaluate_relevance(self, similarity_score: float, threshold: float = 0.35) -> bool:
        # Verifica si el fragmento documental supera el umbral de corte
        return similarity_score >= threshold
```

4.4 Caché Invalidation-Aware con Hash SHA-256 (backend/src/core/cache.py)

```
import hashlib
import time
from typing import Optional, Dict, Any

class InvalidationAwareQueryCache:
    def __init__(self, ttl_seconds: int = 3600):
        # Almacenamiento en memoria con tiempo de expiracion TTL
        self.exact_cache: Dict[str, Dict[str, Any]] = {}
        self.ttl = ttl_seconds
        self.last_docs_hash: str = ""

    def _hash_query(self, query: str) -> str:
        # Genera el resumen criptografico SHA-256 de la consulta normalizada
        normalized = query.strip().lower().encode("utf-8")
        return hashlib.sha256(normalized).hexdigest()

    def get(self, query: str) -> Optional[Dict[str, Any]]:
        # Verificacion de existencia en tiempo constante O(1)
        key = self._hash_query(query)
        if key in self.exact_cache:
            record = self.exact_cache[key]
            # Valida vigencia temporal del registro almacenado
            if time.time() - record["timestamp"] < self.ttl:
                return record["response"]
            del self.exact_cache[key]
        return None

    def set(self, query: str, response: Dict[str, Any]) -> None:
        key = self._hash_query(query)
        self.exact_cache[key] = {
            "response": response,
            "timestamp": time.time()
        }
```

5. Buenas Prácticas de Programación y Calidad de Software

1. Separación de Responsabilidades y Alta Cohesión:

- La capa de presentación (frontend/) desconoce los detalles de persistencia; consume exclusivamente el gateway REST de FastAPI mediante contratos tipados.
- El motor léxico `bm25.py` está desacoplado del motor vectorial `vector_store.py` , permitiendo probar o sustituir cualquiera de los dos sin afectar al orquestador `hybrid_retriever.py` .

2. Tipado Estático y Programación Defensiva:

- Todo el código en Python utiliza anotaciones de tipo estrictas (`typing.List` , `typing.Dict` , `typing.Optional`) validadas en tiempo de ejecución por Pydantic v2.
- En el frontend, TypeScript previene errores de tipo en tiempo de compilación.

3. Manejo Asíncrono no Bloqueante:

- Uso de `async def` y llamadas `await` con `httpx.AsyncClient` para garantizar que la API pueda atender concurrentemente cientos de consultas sin saturar el bucle de eventos (*Event Loop*).

4. Reutilización de Sockets TCP con Connection Pooling:

- Configuración de `httpx.Limits(max_keepalive_connections=20, max_connections=50)` para evitar el costo de apertura de handshakes TCP repetitivos.

5. Cobertura de Pruebas Automatizadas:

- La suite en `backend/tests/` cuenta con 27 pruebas que validan guardrails, lematización BM25, fusión RRF, navegación determinística y contratos de API con un tiempo de ejecución menor a 35 segundos.